

VibeChef – AI-Powered Recipe Generator

Project Report

1. Project Overview

1.1 Project Title

VibeChef – AI-Powered Recipe Generator

1.2 Application Name

VibeChef

1.3 Live Application

<https://vibechef-hm3i.onrender.com/>

1.4 Project Objective

VibeChef is an AI-powered web application designed to help users decide what they can cook using the ingredients available to them.

Users can enter **all the ingredients they currently have**, and the application uses an AI model to process the input and generate a suitable recipe suggestion. The primary goal is to make recipe discovery simple and convenient while helping users make better use of the ingredients already available in their kitchen.

The project demonstrates the use of AI-assisted development, backend API integration, secure environment variables, containerization, and cloud deployment.

2. Problem Statement

Deciding what to cook can be challenging when users have a limited set of ingredients available at home. People may have several ingredients but may not know how to combine them into a suitable dish.

Traditional recipe platforms generally require users to search for a specific recipe or browse through large collections of recipes. This can be time-consuming when the user simply wants to know what they can prepare using the ingredients they already have.

VibeChef addresses this problem by allowing users to provide their available ingredients and receive an AI-generated recipe suggestion based on their input.

The application aims to provide a simple and interactive solution that connects the user's available ingredients with AI-powered recipe generation.

3. Target Users and Use Case

3.1 Target Users

VibeChef is designed for:

- Students who want simple meal ideas.
- Young professionals with limited time to decide what to cook.
- Home cooks who want recipe suggestions based on available ingredients.
- Users who want to make better use of ingredients already present in their kitchen.
- Anyone looking for quick and convenient AI-powered recipe ideas.

3.2 Use Case

The primary use case of VibeChef is ingredient-based recipe generation.

A typical user journey is:

1. The user opens the VibeChef web application.
2. The user identifies the ingredients available to them.
3. The user enters **all available ingredients** into the application.
4. The application sends the ingredient information to the backend.
5. The backend communicates with the integrated AI model.
6. The AI processes the provided ingredients.
7. The generated recipe suggestion is returned to the application.
8. The user views the suggested recipe and uses it as a cooking idea.

Providing a complete list of available ingredients helps the AI generate a more relevant recipe suggestion.

4. Application Features

4.1 Ingredient-Based Recipe Generation

The core functionality of VibeChef is generating recipes based on user-provided ingredients.

Users can enter all the ingredients they currently have, and the AI model processes the information to generate a recipe suggestion.

4.2 AI-Powered Recipe Generation

Instead of depending entirely on a fixed database of recipes, VibeChef uses an external Large Language Model to dynamically generate recipe responses based on the user's input.

This allows the application to provide flexible recipe suggestions for different combinations of ingredients.

4.3 User-Friendly Interface

The application provides a straightforward interface that allows users to interact with the recipe-generation functionality without requiring technical knowledge.

The main interaction is centered around providing ingredients and receiving an AI-generated recipe

4.5 Secure API Key Management

The Groq API key is not directly exposed in the frontend code.

Sensitive credentials are stored using environment variables and accessed by the backend. This helps prevent the API key from being publicly exposed through the client-side application or source code.

4.6 Backend API Integration

The application uses a Node.js and Express.js backend to handle communication between the frontend and the external AI API.

The backend receives the user's request, processes the input, communicates with the AI model, and returns the generated response to the frontend.

4.7 Publicly Accessible Web Application

VibeChef has been deployed online using Render, making the application publicly accessible through a live web URL.

5. Technology Stack

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Node.js
Backend Framework	Express.js
AI Provider	Groq API
AI Model	llama 3.3 70B Versatile
API Communication	Backend API Integration
Environment Configuration	.env Environment Variables
Containerization	Docker
Deployment Platform	Render
Public Access	Live Web URL

6. AI and LLM Integration

VibeChef uses the **Groq API** to connect the application backend with a Large Language Model.

The configured model is:

llama 3.3 70B Versatile

The AI model is responsible for processing the user's ingredient input and generating a recipe suggestion.

The overall flow of the application is:

User Input → Frontend → Backend → Groq API → LLM → Backend → Frontend → Generated Recipe

The API key is stored as an environment variable rather than being included directly in the frontend.

The application environment configuration uses variables such as:

- GROQ_API_KEY
- MONGODB_URI
- GROQ_MODEL
- PORT

Sensitive values are provided through environment configuration and are not exposed publicly.

7. Prompting Strategy and Frameworks Used

Prompt engineering was an important part of the development process because the quality and relevance of the generated recipe depend on how the user's ingredient information is communicated to the AI model.

The application is designed to provide the AI model with the user's available ingredients and request a suitable recipe suggestion.

A representative prompt structure used during development can be described as:

Generate a suitable recipe using the ingredients provided by the user. Consider the complete list of available ingredients and provide a clear recipe suggestion based on them.

The prompting approach focuses on:

- Understanding the user's complete ingredient input.
- Generating a recipe relevant to the provided ingredients.
- Producing a useful response for the user.
- Making the AI response easy to understand.

During development, AI-assisted coding tools were also used to assist with application development, debugging, and implementation.

The project followed a Vibe Coding approach in which AI assistance was used as a development partner while the application structure, functionality, testing, and deployment were reviewed and implemented as part of the project workflow.

8. Vibe Coding Methodology

The application was developed using an AI-assisted development workflow.

The development process involved using AI tools to assist with:

- Project planning.
- Application structure.
- Frontend development.
- Backend development.
- API integration.
- Debugging.
- Docker configuration.
- Deployment troubleshooting.
- Resolving runtime issues.

The development process was iterative. Features were implemented and tested locally before moving toward deployment.

When issues were encountered, the errors were analyzed and the code was updated accordingly.

This approach helped accelerate development while allowing the project to progress through multiple stages from initial concept to a publicly accessible application.

9. Phase-by-Phase Development Summary

Phase 1: Project Ideation

The initial idea was to create an AI-powered application that could help users decide what to cook based on the ingredients they already have.

The concept was named **VibeChef**.

The core idea was kept simple and focused on providing practical value to users.

Phase 2: Frontend Development

The frontend interface was developed to provide users with an easy way to interact with the application.

The interface was designed around the primary workflow:

Entering ingredients → Sending the request → Receiving an AI-generated recipe

HTML, CSS, and JavaScript were used as part of the frontend implementation.

The interface was tested during development to ensure that users could interact with the main functionality.

Phase 3: Backend Development

A Node.js backend using Express.js was implemented to handle API requests.

The backend acts as an intermediary between the frontend and the external AI service.

Instead of exposing the AI API key in the frontend, requests are handled by the backend.

The general backend workflow is:

1. Receive the user's ingredient input.
 2. Process the request.
 3. Communicate with the Groq API.
 4. Send the AI-generated response back to the frontend.
-

Phase 4: AI API Integration

The Groq API was integrated into the backend.

The application was configured to use the **llama 3.3 70B Versatile** model.

Environment variables were used to store sensitive configuration values such as the Groq API key.

This allowed the application to securely communicate with the AI service without placing the API key directly in client-side code.

Phase 5: Testing and Debugging

The application was tested locally during development.

Various issues related to application execution, backend functionality, API integration, and deployment configuration were addressed during the development process.

The application was repeatedly tested to ensure that the main recipe-generation workflow functioned correctly.

Phase 6: Docker Containerization

The application was packaged into a Docker container.

A Dockerfile was created to define the application environment and installation process.

The Docker image was successfully built using the Docker build process.

The application was then run as a Docker container during testing.

This helped create a consistent environment for running the application.

Phase 7: Cloud Deployment

The project was deployed publicly using **Render**.

The deployment process involved connecting the application to the cloud hosting environment and configuring the required environment variables.

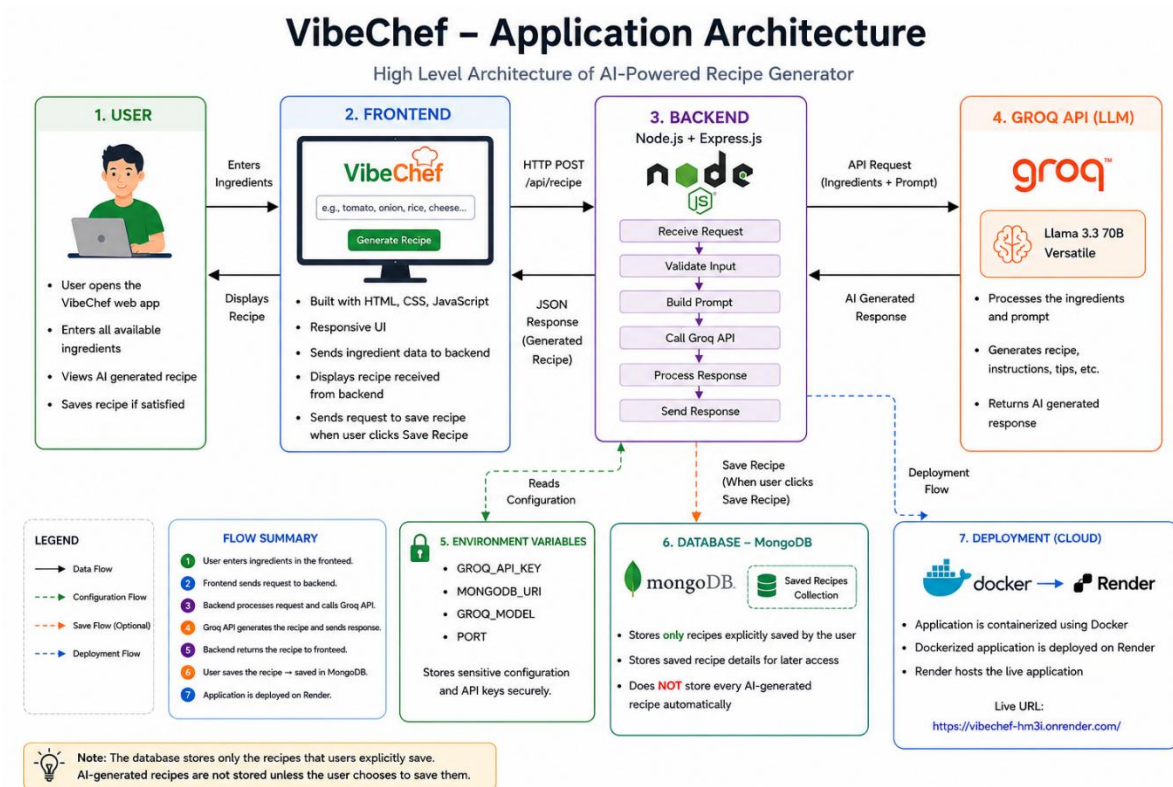
The deployed application became publicly accessible through the live URL:

<https://vibechef-hm3i.onrender.com/>

The live URL was tested and verified to ensure that the deployed application was accessible.

10. Application Architecture

The overall architecture of VibeChef can be represented as follows:



The frontend is responsible for user interaction.

The backend handles application logic and communication with the AI service.

The Groq API provides access to the llama model.

Environment variables are used to protect sensitive credentials.

The application is packaged using Docker and deployed through Render.

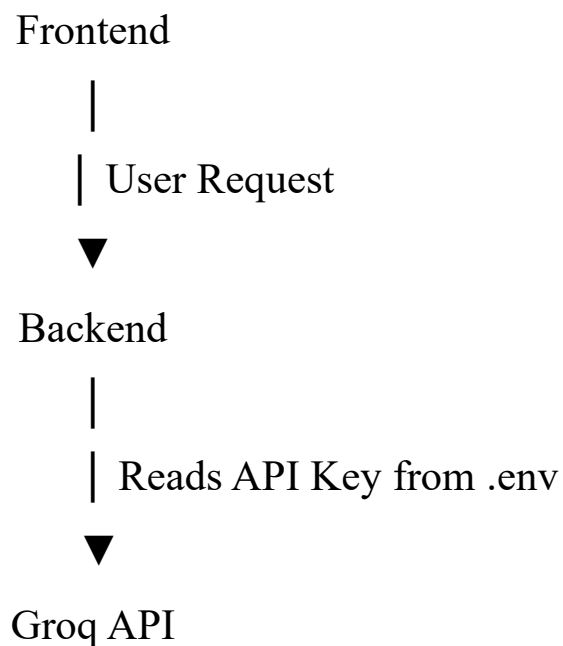
11. Security Implementation

Security was considered during the implementation of the application.

The primary security measure is the protection of the AI API key.

The API key is stored in an environment variable rather than being directly written into frontend JavaScript code.

The general security flow is:



This approach ensures that the API key remains on the server side.

The .env file containing sensitive credentials should not be committed to public version control repositories.

12. Challenges Encountered and Solutions

12.1 Docker Daemon Connection Issue

During the initial Docker setup, the Docker build command could not connect to the Docker API because the Docker Desktop Linux engine was not running.

Solution

The issue was resolved by ensuring that Docker Desktop was running correctly before executing Docker commands.

12.2 Port Conflict

While running the Docker container, port 3000 was already being used by another process or container.

The error indicated that the required port could not be bound because it was already in use.

Solution

The existing process or container using the port was identified and managed before running the application again..

12.4 Cloud Deployment

The application needed to be made publicly accessible so that users could access the project online.

Solution

The application was deployed using Render and configured with the necessary environment variables.

The deployment was successfully completed, and the live application was verified.

13. Deployment

VibeChef is deployed on the Render cloud platform.

The application is publicly accessible through:

<https://vibechef-hm3i.onrender.com/>

The deployment includes the application backend and the required environment configuration for AI API integration.

The live URL was verified after deployment.

14. Key Learnings

The development of VibeChef provided practical experience in several areas of modern web application development.

- ✓ AI-Assisted Development
 - ✓ Full-Stack Application Development
 - ✓ API Integration
 - ✓ Environment Variables
 - ✓ Docker
 - ✓ Cloud Deployment
-

15. Reflection

Developing VibeChef provided an opportunity to experience the complete journey of building an AI-powered web application, from an initial idea to a publicly accessible product.

One of the key learnings from the project was that developing an AI application involves more than simply connecting an AI model to a user interface. The application also requires a functional frontend, a backend that handles API communication, testing, containerization, and deployment.

The project also demonstrated the value of iterative development. Issues encountered during Docker setup and deployment were resolved through debugging and repeated testing.

Overall, the project improved understanding of AI integration, backend development, secure API handling, Docker containerization, and cloud deployment.

16. Future Scope

Although the current version focuses on ingredient-based recipe generation, VibeChef could be extended in the future with additional functionality such as:

- Recipe history .
- User accounts and personalized preferences.
- Nutritional information.
- Ingredient substitution suggestions.
- Shopping list generation.
- More advanced personalization.

These features are considered potential future improvements and are **not part of the current implemented version.**

17. Conclusion

VibeChef is an AI-powered recipe generation web application that helps users discover recipe ideas based on the ingredients they have available.

The project combines a frontend interface, Node.js and Express.js backend, Groq API integration, the llama 3.3 70B Versatile model, environment-based API key security, Docker containerization, and public cloud deployment.

The project demonstrates the practical application of AI-assisted development and Vibe Coding methodologies across the development lifecycle.

The final application is publicly accessible through the following live URL:

<https://vibechef-hm3i.onrender.com/>

The project demonstrates the process of taking an AI-powered application from concept and development through testing, containerization, and public deployment.